Buổi 1

Tổng quan về Cấu trúc dữ liệu và Giải thuật

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Tầm quan trọng của CTDL và GT

1.1 Xây dựng cấu trúc dữ liệu và xây dựng giải thuật
1.2 Mối quan hệ giữa CTDL và giải thuật
1.3 Các cách biểu diễn giải thuật

2. Đánh giá CTDL và giải thuật

2.1 Các tiêu chuẩn đánh giá cấu trúc dữ liệu
2.2 Đánh giá độ phức tạp của thuật toán
2.3 Ký hiệu O lớn và các lớp độ phức tạp
2.4 Luyện tập đếm phép toán và xác định độ phức tạp
2.5 Ví dụ phân tích ba trường hợp: tìm tuyến tính
2.6 Đệ quy và độ phức tạp của giải thuật đệ quy

3. Kiểu dữ liệu

3.1 Khái niệm kiểu dữ liệu và các kiểu dữ liệu cơ sở
3.2 Kiểu dữ liệu có cấu trúc, con trỏ và tập tin
3.3 Kiểu dữ liệu trừu tượng

4. Tổng kết và thực hành

4.1 Tóm tắt và bài thực hành buổi 1
4.2 Bài thực hành buổi 1
4.3 Câu hỏi ôn tập và bài tập

1.1 Xây dựng cấu trúc dữ liệu và xây dựng giải thuật

- **Dữ liệu**: dữ liệu vào (input), dữ liệu trung gian, dữ liệu ra (output). Không có chương trình nào không xử lý dữ liệu.
- **Cấu trúc dữ liệu (CTDL)**: cách **tổ chức, lưu trữ** dữ liệu trong bộ nhớ để phục vụ xử lý. CTDL quyết định rất lớn đến chất lượng chương trình và công sức lập trình.
- **Giải thuật (thuật toán)**: phương pháp, cách thức giải quyết vấn đề – một dãy hữu hạn các bước rõ ràng biến dữ liệu vào thành dữ liệu ra.
- Cách biểu diễn giải thuật: ngôn ngữ tự nhiên, **sơ đồ khối** (flow chart), **mã giả** (pseudo code), ngôn ngữ lập trình (C/C++).

- **Đặc trưng của giải thuật:**
  - **Tính dừng**: kết thúc sau hữu hạn bước.
  - **Tính xác định**: mỗi bước rõ ràng, không mập mờ.
  - **Tính đúng**: cho kết quả đúng với mọi bộ dữ liệu hợp lệ.
  - **Tính phổ dụng**: giải được một lớp bài toán.
  - **Đầu vào/đầu ra** xác định.

Ví dụ 1.1 – Mã giả theo phong cách giáo trình: tìm giá trị lớn nhất của mảng M có N phần tử

```
B1: Max = M[1], k = 2
B2: IF (k > N) Thực hiện Bkt
B3: IF (M[k] > Max) Max = M[k]
B4: k++
B5: Lặp lại B2
Bkt: Kết thúc
```

1.2 Mối quan hệ giữa CTDL và giải thuật

Đẳng thức của N. Wirth

Cấu trúc dữ liệu + Giải thuật = Chương trình

- Có CTDL tốt + nắm vững giải thuật ⇒ cài đặt chỉ còn là vấn đề thời gian. Có CTDL mà chưa có giải thuật – hoặc ngược lại – thì **chưa** có chương trình.
- Chọn CTDL **khác nhau** dẫn tới giải thuật **khác nhau** với hiệu quả rất khác nhau.

Ví dụ 1.2 – Cùng bài toán, hai CTDL: tra cứu số điện thoại trong danh bạ 1.000.000 người

CTDL	Giải thuật tìm	Số phép so sánh xấu nhất	Thêm một người
Mảng **không sắp xếp**	Tìm tuyến tính	1.000.000	Rất nhanh: thêm vào cuối
Mảng **đã sắp xếp** theo tên	Tìm nhị phân	≈ 20 ($\log_2 10^6$)	Chậm: dời trung bình 500.000 phần tử
Cây nhị phân tìm kiếm cân bằng (AVL)	Tìm trên cây	≈ 20	Nhanh: ≈ 20 bước + quay cây

- **Phân tích:** không có CTDL "tốt nhất" tuyệt đối; phải cân nhắc thao tác nào **thường xuyên** (tìm hay thêm/xóa) để chọn CTDL phù hợp. Đây là mạch xuyên suốt môn học (buổi 12 tổng kết lại).

Ví dụ 1.3 – Bảng giờ tàu (bài tập giáo trình)

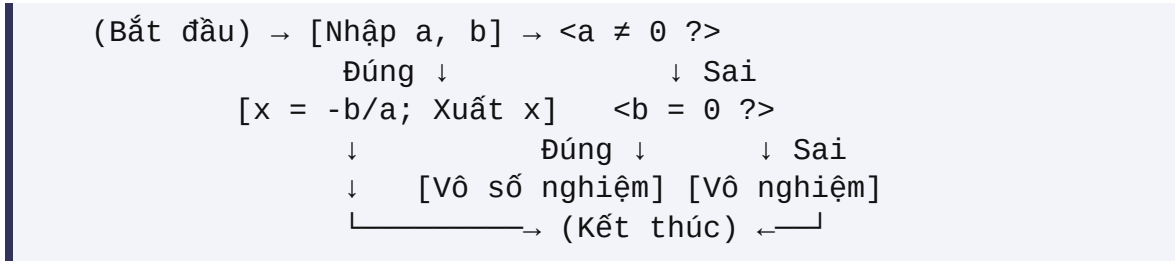
Lưu giờ tàu đến của 11 chuyến tàu tại 23 ga: dùng **mảng 2 chiều** `GioDen[23][11]` (struct {giờ, phút}) ⇒ tra "giờ đến của tàu T0 tại ga G0" chỉ là một phép truy cập `GioDen[G0][T0]` – O(1).

1.3 Các cách biểu diễn giải thuật

- Cùng một giải thuật – **giải phương trình bậc nhất ax + b = 0** – biểu diễn theo 4 cách:
- **Ngôn ngữ tự nhiên:** Nếu a ≠ 0 thì nghiệm x = −b/a. Nếu a = 0: nếu b = 0 thì vô số nghiệm, ngược lại vô nghiệm.
- **Mã giả (theo giáo trình):**

```
B1: Nhập a, b
B2: IF (a ≠ 0)
      B2.1: x = -b/a; Xuất x
      B2.2: Thực hiện Bkt
B3: IF (b = 0) Xuất "Vô số nghiệm"
B4: ELSE Xuất "Vô nghiệm"
Bkt: Kết thúc
```

- **Sơ đồ khối:**



- Ký hiệu: hình **elip** – bắt đầu/kết thúc; hình **chữ nhật** – xử lý; hình **thoi** – điều kiện; hình **bình hành** – nhập/xuất.
- **C/C++:** `if (a != 0) printf("%f", -b/a); else if (b == 0) printf("Vo so nghiem"); else printf("Vo nghiem");`

Lưu ý

Môn học dùng **mã giả B1, B2, …, Bkt** để trình bày ý tưởng rồi mới cài đặt bằng C/C++ – sinh viên nên tập viết mã giả trước khi viết code.

2.1 Các tiêu chuẩn đánh giá cấu trúc dữ liệu

Tiêu chuẩn	Ý nghĩa	Ví dụ vi phạm
Phản ánh đúng thực tế	Biểu diễn đầy đủ, chính xác thông tin của bài toán	Lưu điểm SV bằng kiểu `int` ⇒ mất điểm lẻ 7.5
Phù hợp với các thao tác	Thao tác thường dùng thực hiện dễ dàng, hiệu quả	Danh sách thường xuyên chèn giữa mà dùng mảng ⇒ phải dời nhiều phần tử
Tiết kiệm tài nguyên	Dùng bộ nhớ hợp lý	Khai báo `char HoTen[1000]` cho họ tên ⇒ lãng phí; mảng 10^6 phần tử khi chỉ dùng 100

Ví dụ 1.4 – Chọn CTDL lưu đa thức $fn(x) = \Sigma a_i x^i$, $0 \le n \le 100$ (bài tập giáo trình)

- **Cách 1 – mảng hệ số:**

```
typedef struct {
   int   n;          // bậc
   float a[101];     // a[i]: hệ số của x^i
} DaThuc;
```

- Tính giá trị, cộng, nhân rất đơn giản: cộng hai đa thức chỉ là `c.a[i] = p.a[i] + q.a[i]`.

- **Cách 2 – chỉ lưu số hạng khác 0** (danh sách các cặp (hệ số, số mũ)).
- **Phân tích:** đa thức $5x^{100} + 1$: cách 1 tốn **101** số thực, cách 2 chỉ **2** cặp ⇒ cách 2 tiết kiệm khi đa thức "thưa"; nhưng cách 1 thao tác dễ hơn. Lựa chọn tùy đặc điểm dữ liệu thực tế.

2.2 Đánh giá độ phức tạp của thuật toán

- Thời gian chạy thực tế phụ thuộc máy, trình biên dịch, dữ liệu… ⇒ ta ước lượng **T(n)** theo **kích thước dữ liệu n**, bằng cách **đếm số phép toán cơ bản**: số phép **so sánh** (S) và số phép **gán** (G).
- Xét ba trường hợp: **tốt nhất Tmin**, **xấu nhất Tmax**, **trung bình Tavg**.

Ví dụ 1.5 – Phân tích hàm tìm Max (ví dụ 1.1) với mảng N phần tử

```
int TimMax(int M[], int N)
{  int Max = M[0];                // 1 phép gán
   for (int k = 1; k < N; k++)    // N-1 lần lặp: N phép so sánh k<N, N-1 phép gán k++ (+1 gán k=1)
      if (M[k] > Max)             // N-1 phép so sánh
         Max = M[k];              // 0 .. N-1 phép gán
   return Max;
}
```

Trường hợp	Khi nào	Số phép so sánh	Số phép gán (Max = …)
Tốt nhất	M[0] là lớn nhất	2N − 1	1
Xấu nhất	Mảng tăng dần	2N − 1	N
Trung bình	Dữ liệu ngẫu nhiên	2N − 1	≈ 1 + ln N

- **Kết luận:** số phép so sánh luôn tỉ lệ với N ⇒ TimMax có độ phức tạp **O(N)** trong mọi trường hợp.

2.3 Ký hiệu O lớn và các lớp độ phức tạp

Định nghĩa

T(n) = **O(f(n))** nếu tồn tại hằng số c > 0 và n_0 sao cho **T(n) ≤ c·f(n)** với mọi n ≥ n_0. Nghĩa là T(n) **không tăng nhanh hơn** f(n) khi n lớn.

- **Quy tắc tính nhanh:**
  - Bỏ hằng số: 3n + 5 = O(n).
  - Giữ số hạng bậc cao nhất: $n^2/2 + 100n = O(n^2)$.
  - **Quy tắc cộng**: hai đoạn nối tiếp O(f) và O(g) ⇒ O(max(f, g)).
  - **Quy tắc nhân**: vòng lặp lồng nhau ⇒ O(f·g).

Lớp	Tên	n = 10^6 (≈ số bước)	Ví dụ
O(1)	Hằng	1	Truy cập M[i]
O(log n)	Logarit	20	Tìm nhị phân
O(n)	Tuyến tính	10^6	Tìm tuyến tính
O(n log n)	n log n	$2·10^7$	QuickSort, HeapSort, MergeSort
$O(n^2)$	Bình phương	10^{12}	Bubble, Selection, Insertion
$O(2^n)$	Mũ	không khả thi	Duyệt mọi tập con

Ví dụ 1.6 – Ý nghĩa thực tế (máy thực hiện 10^8 phép toán/giây, n = 10^6)

O(n log n) ≈ 0,2 giây; $O(n^2) ≈ 10^4$ giây ≈ **2,8 giờ**. ⇒ Với dữ liệu lớn, chọn thuật toán đúng lớp quan trọng hơn rất nhiều so với máy nhanh.

Ví dụ 1.7 – Xác định độ phức tạp các đoạn mã

Đoạn mã	Số lần thực hiện lệnh trong cùng	Độ phức tạp
`for (i=0; i<n; i++) s += i;`	n	O(n)
`for (i=0; i<n; i++) for (j=0; j<n; j++) s++;`	n^2	$O(n^2)$
`for (i=0; i<n; i++) for (j=i+1; j<n; j++) s++;`	$(n-1) + \ldots + 1 = n(n-1)/2$	$O(n^2)$
`for (i=1; i<n; i*=2) s++;`	$\lfloor \log_2 n \rfloor + 1$ (i = 1, 2, 4, …)	O(log n)
`for (i=0; i<n; i++) for (j=1; j<n; j*=2) s++;`	$n \cdot \log_2 n$	O(n log n)
`for (i=0; i<n; i++) s++; for (j=0; j<n*n; j++) s++;`	$n + n^2$	$O(n^2)$ (quy tắc cộng)

Ví dụ 1.8 – Phân tích chi tiết dòng 3 (vòng lặp tam giác)

Với i = 0, vòng j chạy n − 1 lần; i = 1: n − 2 lần; …; i = n − 1: 0 lần. Tổng = (n−1) + (n−2) + … + 0 = **n(n−1)/2**. Với n = 1000: 499.500 lần ≈ ½·n^2 ⇒ $O(n^2)$. Đây chính là số phép so sánh của các thuật toán sắp xếp đơn giản (buổi 3, 4).

2.5 Ví dụ phân tích ba trường hợp: tìm tuyến tính

Ví dụ 1.10 – Tìm phần tử X trong mảng M có N phần tử (chuẩn bị cho buổi 2)

```
int LinearSearch(int M[], int N, int X)
{  int k = 0;                        // 1 phép gán
   while (k < N && M[k] != X)        // mỗi vòng: 2 phép so sánh
      k++;                           // mỗi vòng: 1 phép gán
   if (k < N) return k;              // 1 phép so sánh
   return -1;
}
```

Trường hợp	Tình huống	Số phép so sánh S	Số phép gán G
Tốt nhất (Tmin)	M[0] = X	2 + 1 = 3	1
Xấu nhất (Tmax)	Không có X	2N + 1 + 1 = 2N + 2	N + 1
Trung bình (Tavg)	X ở vị trí ngẫu nhiên	≈ (3 + 2N + 2)/2 = N + 2,5	≈ N/2 + 1

- **Phân tích:** Tmax và Tavg đều tỉ lệ với N ⇒ tìm tuyến tính là **O(N)**. Buổi 2 sẽ thấy kỹ thuật **phần tử cầm canh** giảm một nửa số phép so sánh (bỏ điều kiện k < N) nhưng vẫn O(N); còn **tìm nhị phân** giảm xuống O(log N).

Vì sao thường quan tâm Tmax?

Tmax là **cam kết** về thời gian: chương trình không bao giờ chậm hơn mức đó. Tavg cần giả thiết về phân bố dữ liệu, khó tính; Tmin ít có ý nghĩa thực tế.

2.6 Đệ quy và độ phức tạp của giải thuật đệ quy

- **Đệ quy**: hàm gọi lại chính nó với dữ liệu **nhỏ hơn**; gồm **điều kiện dừng** và **bước đệ quy**. Dùng nhiều trong tìm nhị phân, QuickSort, MergeSort, các thao tác trên cây.

Ví dụ 1.11 – Giai thừa

```
long long GiaiThua(int n)
{  if (n <= 1) return 1;        // điều kiện dừng
   return n * GiaiThua(n - 1);  // bước đệ quy
}
```

- T(n) = T(n − 1) + c ⇒ T(n) = c·n ⇒ **O(n)**; bộ nhớ ngăn xếp cũng O(n).

Ví dụ 1.12 – Fibonacci đệ quy "ngây thơ"

`F(n) = F(n−1) + F(n−2)`: số lời gọi tăng theo **cấp số mũ** ≈ $1{,}6^n$ (F(40) cần ≈ $3 \cdot 10^8$ lời gọi). Dùng vòng lặp chỉ còn **O(n)**.

Công thức truy hồi	Nghiệm	Thuật toán điển hình
T(n) = T(n−1) + c	O(n)	Giai thừa, duyệt danh sách đệ quy
T(n) = T(n/2) + c	O(log n)	Tìm nhị phân
T(n) = 2T(n/2) + c·n	O(n log n)	MergeSort, QuickSort (tốt)
T(n) = T(n−1) + c·n	$O(n^2)$	QuickSort (xấu nhất)
T(n) = 2T(n−1) + c	$O(2^n)$	Tháp Hà Nội

- **Giải T(n) = T(n/2) + c** bằng thay thế: T(n) = T(n/4) + 2c = … = T(1) + c·$\log_2 n$ ⇒ O(log n).

3.1 Khái niệm kiểu dữ liệu và các kiểu dữ liệu cơ sở

Kiểu dữ liệu T = <V, O>

Kiểu dữ liệu T là sự kết hợp của **miền giá trị V** mà T có thể lưu trữ và **tập phép toán O** thao tác trên dữ liệu. Số byte cần để lưu một phần tử gọi là **kích thước** của kiểu.

Kiểu cơ sở	Kích thước (C/C++ phổ biến)	Miền giá trị V	Tập phép toán O
Số nguyên: char, short, int, long	1, 2, 4, 4/8 byte	int: -2^{31} … $2^{31}-1$	+, −, *, /, %, <, >, ==, …
Số thực: float, double	4, 8 byte	float ≈ 7 chữ số có nghĩa	+, −, *, /, so sánh
Ký tự: char	1 byte	mã ASCII 0 … 255	so sánh, chuyển mã
Luận lý: bool	1 byte	true, false	NOT (!), AND (&&), OR (\ | \ |)

Ví dụ 1.9 – Hậu quả của việc chọn kiểu sai

- `int` tính giai thừa: 13! = 6.227.020.800 > $2^{31}-1$ ⇒ **tràn số**, kết quả sai mà không báo lỗi ⇒ dùng `long long` (đến 20!).
- `float` cộng 0.1 mười lần ≠ 1.0 do sai số biểu diễn ⇒ không so sánh số thực bằng `==`, dùng `fabs(a - b) < 1e-6`.
- `unsigned int i; for (i = n - 1; i >= 0; i--)` ⇒ **vòng lặp vô hạn** vì i không bao giờ âm.

3.2 Kiểu dữ liệu có cấu trúc, con trỏ và tập tin

- **Mảng**: dãy phần tử cùng kiểu, truy cập theo chỉ số O(1); kích thước = tổng kích thước các phần tử.
- **Cấu trúc (struct / bản ghi)**: gom các thành phần (field) khác kiểu.

```
typedef struct {
   char  MaSV[11];
   char  HoTen[31];
   float DTB;
} SinhVien;              // ≈ 11 + 31 + 4 = 46 byte (+ căn
lề)
SinhVien DS[100];        // mảng các struct
```

- **Con trỏ**: lưu **địa chỉ** bộ nhớ; nền tảng của danh sách liên kết, cây (buổi 7–12).

```
int x = 5;  int *p = &x;  *p = 10;  // x = 10
SinhVien *q = new SinhVien;  q->DTB = 8.5;  delete q;
```

- Giáo trình (16-bit) phân biệt con trỏ gần 2 byte / xa 4 byte; máy 64-bit hiện nay: con trỏ **8 byte**.
- **Tập tin**: dữ liệu trên bộ nhớ ngoài; kích thước chỉ giới hạn bởi đĩa; truy xuất **chậm** hơn RAM hàng nghìn lần ⇒ thường nạp từng phần vào bộ nhớ trong để xử lý (cơ sở của **tìm kiếm ngoại**, **sắp xếp ngoại** – buổi 2, 5).

```
FILE *f = fopen("SV.DAT", "rb");
SinhVien sv;
while (fread(&sv, sizeof(SinhVien), 1, f) == 1)
    printf("%s %.1f\n", sv.HoTen, sv.DTB);
fclose(f);
```

3.3 Kiểu dữ liệu trừu tượng (ADT)

Kiểu dữ liệu trừu tượng

Là kiểu dữ liệu chỉ được mô tả bằng **tập giá trị và các phép toán** (cái gì), **không** quy định cách cài đặt (làm thế nào). Ví dụ: Danh sách, Ngăn xếp, Hàng đợi, Cây – các ADT sẽ học từ buổi 6.

Ví dụ 1.13 – ADT Ngăn xếp (Stack)

- **V**: dãy phần tử, thêm/lấy ở **một đầu** (đỉnh).
- **O**: Initialize, IsEmpty, IsFull, Push(x), Pop(), Top().
- **Hai cách cài đặt** (buổi 9): bằng **mảng** hoặc bằng **danh sách liên kết** – chương trình dùng stack không cần biết cách nào.

Kiểu	Thuộc nhóm	Cài đặt bởi
int, float, char	Cơ sở	Ngôn ngữ cung cấp sẵn
Mảng, struct	Có cấu trúc	Ngôn ngữ cung cấp sẵn
Danh sách, Stack, Queue, Cây	ADT	Người lập trình xây dựng từ mảng/struct/con trỏ

- **Lợi ích**: tách biệt "sử dụng" và "cài đặt" ⇒ dễ thay đổi cách cài đặt mà không sửa chương trình gọi (giống tính độc lập dữ liệu trong CSDL).

4.1 Tóm tắt và bài thực hành buổi 1

- **CTDL + Giải thuật = Chương trình**; CTDL tốt phải phản ánh đúng thực tế, phù hợp thao tác, tiết kiệm tài nguyên.
- Đánh giá thuật toán bằng **số phép so sánh, phép gán** trong trường hợp tốt nhất / xấu nhất / trung bình; biểu diễn bằng **ký hiệu O**.
- Kiểu dữ liệu T = <V, O>: kiểu cơ sở, có cấu trúc (mảng, struct), con trỏ, tập tin; kiểu dữ liệu trừu tượng (ADT).
- Giải thuật đệ quy: lập công thức truy hồi để ước lượng độ phức tạp.

Khái niệm	Câu hỏi cần trả lời
CTDL	Dữ liệu được tổ chức thế nào trong bộ nhớ?
Giải thuật	Các bước xử lý dữ liệu là gì? Có dừng, đúng không?
Độ phức tạp	Khi n tăng gấp đôi, thời gian tăng bao nhiêu lần?
ADT	Những phép toán nào được phép? (không quan tâm cài đặt)

Thực hành – Đếm phép so sánh/gán và đo thời gian chạy

```
#include <stdio.h>
#include <time.h>
long SoSanh = 0, Gan = 0;
int TimMax(int M[], int N) {
    int Max = M[0]; Gan++;
    for (int k = 1; k < N; k++) {
        SoSanh++;
        if (M[k] > Max) { Max = M[k]; Gan++; }
    }
    return Max;
}
int main() {
    static int M[1000000];
    for (int i = 0; i < 1000000; i++) M[i] = rand();
    clock_t t = clock();
    TimMax(M, 1000000);
    printf("So sanh: %ld, Gan: %ld, Thoi gian: %.3f s\n", SoSanh, Gan, (double)(clock()-t)/CLOCKS_PER_SEC);
}
```

- Yêu cầu: chạy với N = 10^3, 10^4, 10^5, 10^6 cho mảng **ngẫu nhiên**, **tăng dần**, **giảm dần**; ghi bảng kết quả và đối chiếu với bảng phân tích ở ví dụ 1.5.

4.3 Câu hỏi ôn tập và bài tập

Câu hỏi

- 1) Trình bày tầm quan trọng của CTDL và GT đối với người lập trình. Khi xây dựng giải thuật có cần quan tâm tới CTDL không? Vì sao?
- 2) Nêu các tiêu chuẩn đánh giá CTDL; cho ví dụ một CTDL vi phạm từng tiêu chuẩn.
- 3) Phân biệt Tmin, Tmax, Tavg; vì sao người ta thường quan tâm Tmax?
- 4) Sắp xếp theo thứ tự tăng dần tốc độ tăng: n^2, log n, n log n, 2^n, n, 1, n^3.

Bài tập (có gợi ý)

- 1) Xác định độ phức tạp: `for (i = n; i > 0; i /= 2) for (j = 0; j < i; j++) s++;` → **Gợi ý:** tổng n + n/2 + n/4 + … ≤ 2n ⇒ **O(n)** (không phải O(n log n)!).
- 2) Xây dựng CTDL lưu đa thức bậc n ≤ 100; viết hàm nhập, xuất, tính giá trị tại x_0 (gợi ý: dùng **sơ đồ Horner** để chỉ cần n phép nhân), cộng, nhân hai đa thức.
- 3) Tương tự bài 2 nhưng hệ số là **phân số** (tử, mẫu nguyên): định nghĩa `struct PhanSo` và các phép toán.
- 4) Bảng giờ tàu (giáo trình): xây dựng CTDL lưu trên bộ nhớ trong và tập tin; viết hàm cho biết giờ đến của tàu T0 tại ga G0, thời gian tàu T0 đi từ ga G1 đến G2.